

TECHNICAL COMPARISON

Tropical Cyclone Intensity Prediction Models

Notebook A (Model which is built with Full Dataset) vs Notebook B-(Model with small sample dataset)

1. Overview

This document provides a side-by-side technical comparison of two Jupyter notebooks developed for tropical cyclone intensity prediction, both trained on the TCIR (Tropical Cyclone Image Repository) dataset. **Notebook A** (`Tropical_cyclone_intensity_prediction.ipynb`) is a production-grade pipeline covering the full multi-basin TCIR dataset, while **Notebook B** (`tropical_cyclone_prediction_with_CNN.ipynb`) is a focused baseline pipeline using a single regional file. Each notebook trains both a CNN and a ConvLSTM model; however their design philosophies, data handling, and model sophistication differ substantially.

Attribute	Notebook A (Full Pipeline)	Notebook B (Baseline)
Primary Goal	Multi-basin production system	Baseline exploration / learning
Dataset Scope	3 TCIR tar.gz files (all basins)	1 TCIR .h5 file (CPAC/IO/SH)
Image Resolution	64 × 64 px (resized)	201 × 201 px (original)
Data Split Strategy	Storm-wise (no leakage)	Random (potential leakage)
Loss Function	Huber (robust to outliers)	MSE (sensitive to outliers)
Memory Strategy	Chunked loading + float16	Full in-memory float32
Bias Analysis	Yes (intensity-binned)	Not included

2. Data Pipeline Comparison

2.1 Dataset Loading & Scope

Notebook A mounts three tar.gz archives from Google Drive — `TCIR-ALL_2017.h5`, `TCIR-ATLN_EPAC_WPAC.h5`, and `TCIR-CPAC_IO_SH.h5` — covering all major tropical cyclone basins globally. The extraction is conditional: if all three .h5 files already exist in the target directory, re-extraction is skipped. This design supports iterative development without redundant I/O. In contrast, **Notebook B** loads only the single regional file `TCIR-CPAC_IO_SH.h5` via `pd.read_hdf()` and `h5py`, ingesting the entire 201×201×4 matrix into RAM as float32.

2.2 Memory Management

Notebook A employs a rigorous chunked loading strategy inside `load_single_h5_efficient()`. Images are processed in configurable chunk sizes (default 500), bad labels are filtered before any image reads, channel medians are estimated from only the first 500 samples, and all output arrays are stored as float16. This reduces peak RAM by roughly 50% compared to float32. After each chunk, explicit garbage collection via `gc.collect()` is invoked. The combined dataset from all three files is then merged in a single pass.

Notebook B uses no chunking. The entire matrix is loaded at once with `mat = hf['matrix'][:]` into a float32 array. Normalisation is applied in-place via `x /= 255.0`. This approach is simpler but risks memory exhaustion for large datasets. There is no dynamic memory monitoring or garbage collection.

2.3 NaN Handling & Normalisation

Step	Notebook A	Notebook B
NaN Imputation	Per-channel median (sampled from first 500 rows)	Per-channel median (full dataset scan)
Clip Outliers	Yes — 1st/99th percentile per image per channel	No clipping applied
Normalisation	Implicit via percentile clipping	Global divide by 255.0
Label Filtering	5–200 kt hard bounds pre-load	No label filtering

The percentile-based per-image clipping in Notebook A is substantially more robust than the global divide-by-255 in Notebook B. Satellite radiance values in TCIR are not bounded by 255, so the Notebook B normalisation may produce inconsistent scales across channels and across storms.

2.4 Train / Validation / Test Split

Notebook A implements a strict **storm-wise split**. Unique storm IDs are divided 70/10/20 (train/val/test) using `train_test_split` on the storm ID set, then samples belonging to each storm are assigned accordingly. This prevents temporal data leakage — frames from the same storm never appear across sets — which is critical for time-series evaluation integrity.

Notebook B applies a standard **random sample-level split** (80/20) directly on individual frames. Because consecutive frames from the same storm may end up in both train and test, this inflates performance metrics and does not reflect true generalisation to unseen storms.

3. CNN Architecture Comparison

3.1 Architecture Side-by-Side

Component	Notebook A — CNN	Notebook B — CNN
Input Shape	$64 \times 64 \times 4$	$201 \times 201 \times 4$
Conv Block 1	Conv(32) + Conv(32) + BN + MaxPool	Conv(32) + MaxPool
Conv Block 2	Conv(64) + Conv(64) + BN + MaxPool	Conv(64) + MaxPool
Conv Block 3	Conv(128) + BN + MaxPool	Conv(128) + MaxPool
Pooling	GlobalAveragePooling2D	GlobalAveragePooling2D
Dense Head	Dense(128) + Dropout(0.3) + Dense(1)	Dense(64) + Dense(1)
Loss	Huber	MSE
Optimizer	Adam (lr=1e-3)	Adam (lr=5e-4)
Batch Normalisation	Yes (3 layers)	No

Notebook A's CNN includes double convolution blocks (two conv layers before each pooling step) with Batch Normalisation after every block. This is aligned with the VGG-inspired pattern and provides stronger regularisation. The use of Huber loss rather than MSE makes the model more robust to extreme Vmax values (e.g., major hurricanes), which are rare but important outliers in the label distribution.

Notebook B's CNN uses single convolution per block without batch normalisation, and trains with MSE. The larger 201×201 input retains full spatial resolution but significantly increases computation and memory per batch. The Dense head is smaller (64 units vs 128), which may limit capacity.

3.2 Training Configuration

Parameter	Notebook A — CNN	Notebook B — CNN
Max Epochs	60	40
Batch Size	32	16
EarlyStopping	patience=10, min_delta=0.2, monitor val_mae	Defined but commented out
ReduceLROnPlateau	Yes — factor 0.5, patience 5	Not used
TerminateOnNaN	Yes	No
Data Feeding	Custom Keras Sequence generator	In-memory numpy arrays

The absence of active early stopping in Notebook B means training runs the full 40 epochs regardless of convergence, risking overfitting. Notebook A's callbacks form a robust training regime: EarlyStopping prevents overfitting, ReduceLROnPlateau enables fine convergence, and TerminateOnNaN guards against unstable gradients.

4. ConvLSTM Architecture Comparison

4.1 Sequence Construction

Both notebooks build 4-frame sequences from consecutive storm observations (`seq_len = 4`), targeting the Vmax value at the final frame. The key difference is in implementation efficiency. **Notebook A** implements `TCIRDataGenerator`, which indexes into the shared `x_data` memory block on-the-fly, never duplicating the array. **Notebook B** materialises all sequences upfront via `build_storm_sequences()`, creating a new `x_seq` array of shape $(N, 4, H, W, C)$. For the full dataset, this would require several GB of additional RAM. Notebook B partially mitigates this with an 8,000-sample truncation (`N_small = 8000`), but this reduces coverage dramatically.

4.2 ConvLSTM Architecture Side-by-Side

Component	Notebook A — ConvLSTM	Notebook B — ConvLSTM
Input Shape	$4 \times 64 \times 64 \times 4$	$4 \times 201 \times 201 \times 4$ (truncated)
ConvLSTM Layer 1	ConvLSTM2D(32, return_seq=True) + BN	ConvLSTM2D(16, return_seq=True) + BN
ConvLSTM Layer 2	ConvLSTM2D(64, return_seq=False) + BN	ConvLSTM2D(16, return_seq=False) + BN
Post-LSTM Conv	None	Conv2D(32) + MaxPool + Conv2D(64)
Pooling	MaxPooling2D + GlobalMaxPooling2D	GlobalAveragePooling2D
Dense Head	Dense(128) + Dropout(0.4) + Dense(1)	Dense(32) + Dropout(0.3) + Dense(1)
Loss	Huber (lr=5e-4)	MSE (lr=1e-3)
Training Samples	Full dataset, storm-wise	~8,000 samples (truncated)

Notebook A's ConvLSTM doubles the filter counts (32→64) between the two recurrent layers, increasing its temporal feature capacity. Notebook B uses 16 filters in both layers but adds a 2D convolutional head after the recurrent stack — an unusual design that applies spatial convolution on the pooled LSTM output. Both architectures use Batch Normalisation after recurrent layers. Notebook A's use of GlobalMaxPooling vs Notebook B's GlobalAveragePooling can preserve sharper activation peaks (e.g., eye-wall signatures), which may be beneficial for intensity estimation.

5. Evaluation & Post-Training Analysis

5.1 Evaluation Metrics

Metric	Notebook A	Notebook B
MAE (knots)	Reported for both CNN and ConvLSTM	Reported for CNN and ConvLSTM
RMSE (knots)	Computed for both models	Computed for both models
MSE	Reported (via Huber eval)	Reported
R ² Score	Yes	Not computed in final evaluation
Bias Analysis	Yes — mean, std, over/under-prediction %	Not included
Intensity-Binned Error	Yes — 4 bins (TS, Cat1, Cat2-3, Major)	Not included
Scatter Plots	Yes — both models, side-by-side	Yes — ConvLSTM only
Training Curves	Yes — loss + MAE for both models	Yes — loss + MAE for ConvLSTM
Bias Correction	Yes — debiased predictions computed	Not included

Notebook A's bias analysis adds significant scientific value. It reports not only global metrics but also intensity-binned bias (Tropical Storm, Cat1, Cat2-3, Major), and applies bias correction to predictions. This reveals systematic model tendencies (e.g., under-prediction at high intensities) that are invisible in aggregate metrics alone. Notebook B's evaluation is standard but adequate for a baseline exploration.

5.2 Classification Extension

Notebook A includes a tropical cyclone occurrence prediction component derived from the regression outputs. Using a 34-knot threshold (`CYCLONE_THRESHOLD = 34.0`), predicted Vmax values are binarised to classify whether a sample is a tropical cyclone, and classification metrics (precision, recall, F1) are computed. This dual regression+classification capability substantially broadens the system's utility for operational forecasting. Notebook B contains no classification component.

6. Strengths & Limitations

6.1 Notebook A — Full Pipeline

Aspect	Strengths	Limitations
Data Handling	Multi-basin, chunked load, storm-wise split	Higher setup complexity; requires Drive with 3 tar.gz
Model Design	Deeper CNN, double ConvLSTM, Huber loss, BN	Smaller 64×64 input loses fine spatial detail
Training	Full callback suite, memory-safe generators	Longer runtime; requires GPU

Aspect	Strengths	Limitations
Evaluation	Bias analysis, binned errors, classification	No ablation study or uncertainty quantification
Reproducibility	Conditional extraction, random seeds	Cloud-only (Colab + Drive dependent)

6.2 Notebook B — Baseline CNN

Aspect	Strengths	Limitations
Data Handling	Simple, readable, full 201×201 resolution	Single basin; random split causes data leakage
Model Design	Clear, well-commented architecture	No BN; MSE loss; weaker regularisation
Training	Easy to follow; fast on small data	EarlyStopping disabled; ConvLSTM truncated to 8k
Evaluation	Adequate baseline metrics	No R^2 , no bias analysis, no classification
Reproducibility	Minimal dependencies	Normalisation by 255 is incorrect for radiance data

7. Conclusion

For dissertation or research use, **Notebook A is the preferred pipeline**. It implements methodologically sound practices: storm-wise splits prevent data leakage, Huber loss handles intensity outliers robustly, and the bias analysis provides scientifically meaningful insight into model behaviour across the Saffir-Simpson scale. Its memory-efficient design makes it scalable to the complete TCIR multi-basin corpus.

Notebook B serves well as an exploratory baseline or for teaching purposes. Its code is clean, commented, and easy to extend. However, the random data split, disabled early stopping, and truncated ConvLSTM training limit its reliability as an evaluation benchmark.